

BAB V

FUNCTION DAN METHOD

5.1 Tujuan

1. Praktikan mampu mengaitkan konsep abstraksi dan enkapsulasi dalam pemrograman berorientasi objek melalui pemanfaatan *method* secara optimal.
2. Praktikan dapat menganalisis perbedaan arsitektur alur kerja antara subrutin mandiri (*function*) dan subrutin terikat objek (*method*) saat diimplementasikan ke dalam program nyata.
3. Praktikan mampu merancang struktur program yang bersih (*clean code*) dengan menerapkan prinsip modularitas guna menghindari duplikasi baris kode yang redundan.
4. Praktikan dapat mengevaluasi kelebihan serta potensi kendala performa (*performance overhead*) dari penggunaan *method* yang saling bergantung dalam sebuah proyek berskala besar.
5. Praktikan terampil mengintegrasikan penggunaan *access modifier*, parameter masukan, dan tipe data pulangan (*return type*) yang tepat lintas bahasa pemrograman.

5.2 Dasar Teori

5.2.1 Function

Function (fungsi) adalah blok kode yang dirancang untuk melakukan tugas tertentu dan dapat dipanggil berkali-kali dalam program. *Function* memungkinkan *programmer* membagi kode menjadi segmen-segmen kecil yang lebih terkelola, masing-masing melakukan bagian tertentu dari tugas yang lebih besar. Tidak hanya membantu dalam mengorganisasi kode secara lebih efisien, hal ini juga memudahkan pemeliharaan dan pengujian kode.

Setiap *function* biasanya memiliki nama unik dan bisa menerima *input* dalam bentuk parameter, sehingga dapat bekerja dengan data yang berbeda-beda setiap kali dipanggil. *Function* juga bisa mengembalikan nilai sebagai *output*, memungkinkan hasil dari operasinya dipakai di tempat lain dalam program. Struktur ini membantu mengurangi redundansi kode, karena *function* yang sama bisa dipakai kembali di berbagai bagian program tanpa perlu menulis ulang. *Function* memiliki beberapa kegunaan utama yaitu sebagai berikut:

1. **Penggunaan ulang kode:** *function* bisa menggunakan kembali blok kode yang sama tanpa harus menulis ulang. Hal ini berguna dalam kasus di mana ada tugas sama yang perlu dilakukan berkali-kali dalam program.
2. **Menyederhanakan instruksi:** dengan *function*, instruksi yang kompleks dapat disederhanakan menjadi panggilan *function* tunggal. Proses ini membuat kode lebih mudah dibaca dan dipahami.
3. **Mengurangi bug:** *function* membantu mengurangi *bug* karena memudahkan *testing* kode. Jika ada kesalahan dalam *function*, *programmer* hanya perlu memperbaikinya di satu tempat, dan perbaikan tersebut akan berlaku di mana pun *function* itu dipanggil.
4. **Menyembunyikan kompleksitas:** *function* dapat menyembunyikan detail implementasi yang kompleks, sehingga *user* bisa fokus pada apa yang dilakukan *function* tanpa perlu tahu bagaimana hal itu dilakukan.
5. **Memecah kode menjadi bagian lebih kecil:** *function* memudahkan dalam pemeliharaan dan pengembangan lebih lanjut. Kode dapat dibagi

menjadi modul-modul yang berbeda dengan fungsi berbeda, kemudian diimpor dan digunakan di modul utama.

Sumber:

<https://www.revou.co/kosakata/function>

5.2.2 Method

Dalam Pemrograman Berorientasi Objek atau PBO, seperti pada bahasa Java, Python, dan C++, *method* merupakan fungsi yang berada di dalam sebuah kelas (*class*). *Method* digunakan untuk menjalankan suatu tugas atau tindakan tertentu yang berhubungan dengan objek. Sebagai contoh, terdapat sebuah *class* Kucing. Kelas tersebut dapat memiliki *method* seperti *berlari()* atau *makan()*. *Method* tersebut menggambarkan tindakan yang dapat dilakukan oleh objek yang berasal dari kelas Kucing.

Perbedaan antara *method* dan fungsi terletak pada tempat dan cara penggunaannya. Fungsi dapat dibuat secara terpisah dan dipanggil secara langsung menggunakan nama fungsi tersebut. Sebagai contoh, fungsi dapat dipanggil dengan menuliskan *cetak()*. Sementara itu, *method* berada di dalam sebuah kelas dan biasanya dipanggil melalui objek yang dibuat dari kelas tersebut. Pemanggilan *method* umumnya menggunakan operator titik. Sebagai contoh, jika terdapat objek bernama *printer*, maka *method* dapat dipanggil dengan menuliskan *printer.cetak*. Dengan demikian, fungsi lebih bersifat umum, sedangkan *method* biasanya digunakan untuk menjalankan tindakan yang berkaitan dengan objek atau kelas tertentu.

Sumber:

<https://medium.com/@fahmisajid8/perbedaan-method-dan-function-9c559ec6b06>

5.2.3 Penggunaan Function

Sebuah fungsi perlu memiliki struktur yang jelas agar dapat digunakan dengan benar. Secara umum, fungsi terdiri dari beberapa bagian, yaitu tipe data pengembalian, nama fungsi, parameter, dan badan fungsi.

1. Tipe Pengembalian (*Return Type*)

Menunjukkan jenis data yang akan dikembalikan oleh fungsi setelah selesai dijalankan. Contohnya adalah *int*, *float*, atau *string*. Jika fungsi tidak mengembalikan nilai, dapat digunakan tipe *void*.

2. Nama Fungsi (*Function Name*)

Merupakan nama yang digunakan untuk memanggil fungsi. Nama fungsi sebaiknya dibuat sesuai dengan tugas yang dilakukan agar lebih mudah dipahami.

3. Parameter (*Parameter List*):

Parameter merupakan variabel yang digunakan untuk menerima data dari bagian program yang memanggil fungsi. Parameter dapat berjumlah lebih dari satu atau tidak ada sama sekali, tergantung kebutuhan fungsi.

4. Badan Fungsi (*Function Body*):

Berisi perintah-perintah yang akan dijalankan ketika fungsi dipanggil. Pada C++, badan fungsi ditulis di dalam tanda kurung kurawal {}.

Mekanisme Pemanggilan Fungsi Perintah yang terdapat di dalam suatu fungsi akan dijalankan ketika fungsi tersebut dipanggil. Pemanggilan dilakukan dengan menuliskan nama fungsi beserta argumen yang diperlukan. Ketika fungsi dipanggil, nilai argumen akan diberikan kepada parameter yang sesuai. Selanjutnya, program menjalankan perintah yang terdapat di dalam badan fungsi.

Setelah seluruh perintah dalam fungsi selesai dijalankan, program akan kembali melanjutkan proses dari bagian yang melakukan pemanggilan fungsi tersebut. Jika fungsi memiliki nilai yang dikembalikan, nilai tersebut dapat digunakan kembali oleh bagian program yang memanggilnya. Pada program C++, fungsi *main()* memiliki peran khusus karena menjadi titik awal eksekusi program. Ketika program dijalankan, sistem akan memulai proses dari fungsi *main()*.

sumber:

<https://www.scribd.com/document/426169825/KD-3-9-Penggunaan-Fungsi>

5.2.4 Penggunaan Method

Berdasarkan cara penggunaannya, *method* dapat dibedakan menjadi beberapa jenis. Dua jenis yang umum digunakan adalah *instance method* dan *static method*.

1. Instance Method:

Instance method merupakan *method* yang digunakan melalui objek dari suatu kelas. Oleh karena itu, objek harus dibuat terlebih dahulu sebelum *method* tersebut dapat dipanggil. Pada bahasa Python, *instance method* umumnya menggunakan parameter *self* untuk merujuk pada objek yang sedang digunakan.

2. Static Method:

Static method merupakan *method* yang dapat dipanggil tanpa membuat objek dari kelas terlebih dahulu. *Method* ini biasanya dipanggil menggunakan nama kelas dan digunakan untuk proses yang tidak bergantung pada data dari suatu objek tertentu.

Penggunaan *method* memiliki beberapa kelebihan dalam pengembangan program. Salah satunya adalah meningkatkan penggunaan kembali kode atau *reusability*. *Method* yang telah dibuat dapat dipanggil kembali ketika diperlukan sehingga programmer tidak perlu menulis kode yang sama berulang kali. Selain itu, penggunaan *method* dapat meningkatkan keterbacaan atau *readability* program karena setiap *method* dapat dibuat untuk menjalankan tugas tertentu. Struktur program juga menjadi lebih teratur sehingga lebih mudah dipahami dan diperbaiki.

Namun, penggunaan *method* juga memiliki kekurangan apabila tidak dirancang dengan baik. Terlalu banyak *method* atau hubungan yang terlalu rumit antarbagian program dapat membuat kode lebih sulit dipahami. Ketergantungan yang berlebihan antar-*method* juga dapat menyebabkan perubahan pada satu bagian memengaruhi bagian lainnya. Oleh karena itu, *method* perlu dibuat sesuai dengan kebutuhan dan memiliki tugas yang jelas agar struktur program tetap sederhana, teratur, dan mudah dikembangkan.

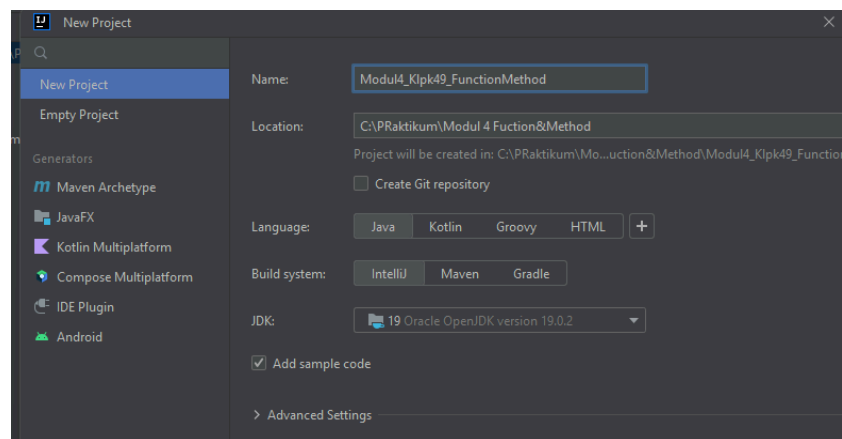
sumber:

<https://www.domainesia.com/berita/method-adalah/>

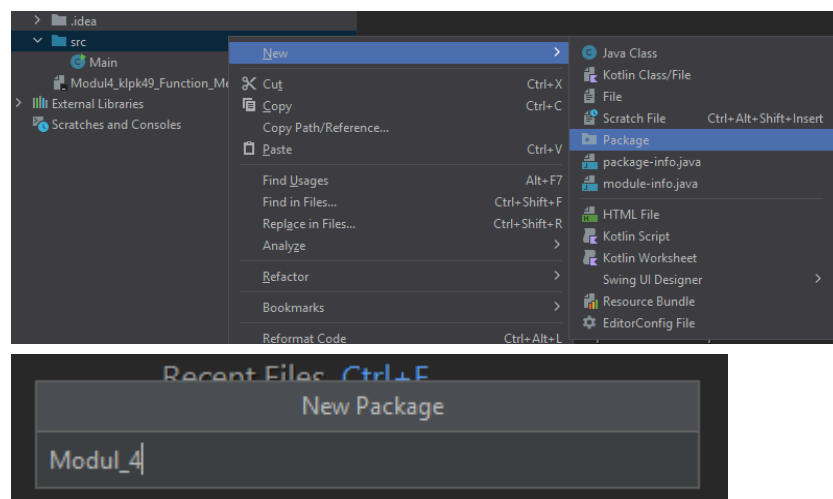
5.3 Langkah Kerja

5.3.1 Function pada Java

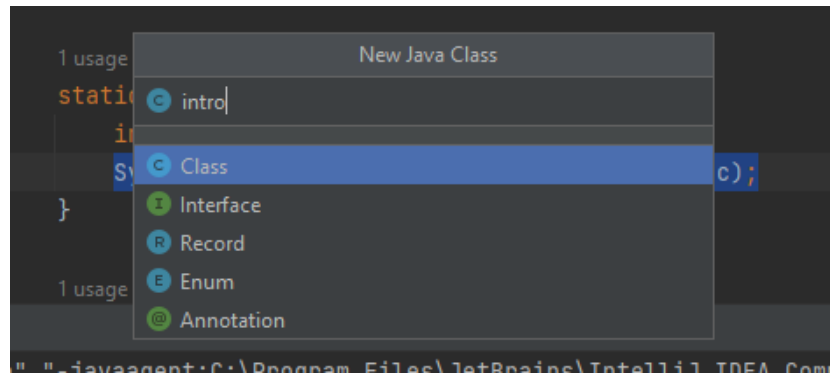
1. Buka *Intellej DEA Community* untuk membuat proyek *Java*
2. Kemudian pilih *new project (file -> new project)*
3. Pilih *java* dan *Intellij* kemudian beri nama *project* dan klik *create*.



4. Selanjutnya, pada bagian *project* (sebelah kiri) klik *button +* atau klik kanan pada *folder src*, kemudian pilih *new*, setelah itu pilih *package*.



5. Setelah itu klik *button +* atau klik kanan pada *package* yang sudah dibuat kemudian pilih *new*, setelah itu pilih *Java class*.
6. Buat sesuai gambar dan sesuaikan nama kelompok masing masing.

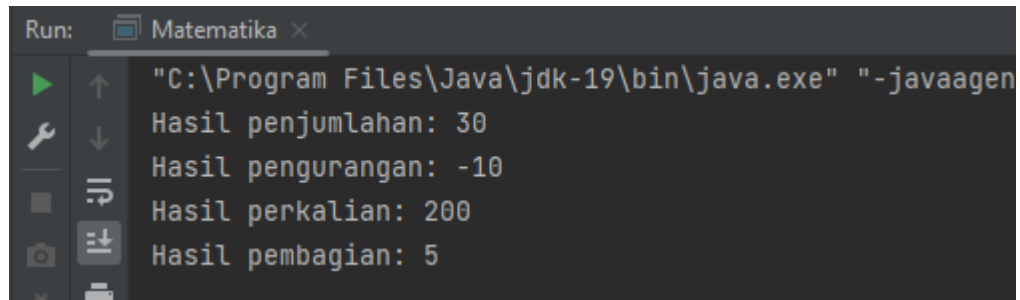


7. Lalu buka *java class* matematika.

8. Masukkan *Source Code* ini

```
public class matematika {  
  
    static void penjumlahan(int a, int b) {  
        int c = a + b;  
        System.out.println("Hasil penjumlahan: " + c);  
    }  
  
    static void pengurangan() {  
        int c = 18 - 24;  
        System.out.println("Hasil pengurangan: " + c);  
    }  
  
    static int pembagian(int a, int b) {  
        int c = a / b;  
        return c;  
    }  
  
    static int perkalian() {  
        int c = 18 * 24;  
        return c;  
    }  
  
    public static void main(String[] args) {  
        penjumlahan(18,24);  
  
        pengurangan();  
  
        System.out.println("Hasil perkalian: " + perkalian());  
  
        int d = pembagian(180,24);  
        System.out.println("Hasil pembagian: " + d);  
    }  
}
```

9. Hasil *Run* program



```
Run: Matematika x
"C:\Program Files\Java\jdk-19\bin\java.exe" "-javaagen
Hasil penjumlahan: 30
Hasil pengurangan: -10
Hasil perkalian: 200
Hasil pembagian: 5
```

5.3.2 Method pada Java

1. Masukkan *Source Code* berikut ke *java class intro* yang sudah dibuat

```
public class intro {

    public void sapa() {
        System.out.println("Haloo Worldd!!");
    }

    public String perkenalan(String nama, String kota, String
hobi) {
        return "Namaku " + nama + ", Aku dari " + kota + "
dan hobiku adalah " + hobi;
    }

    public void umur(int umur) {
        System.out.println("Aku berumur " + umur + " tahun");
    }
}
```

2. Lalu tambahkan code berikut dengan matematika

```
System.out.println();
System.out.println("-----");

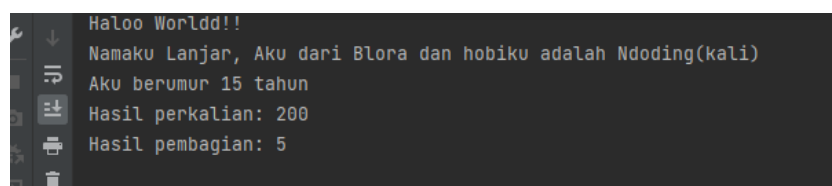
intro objek = new intro();

objek.sapa();

System.out.println(objek.perkenalan("Lanjar", "Blora", "Ndoding
(kali)"));

objek.umur(15);
```

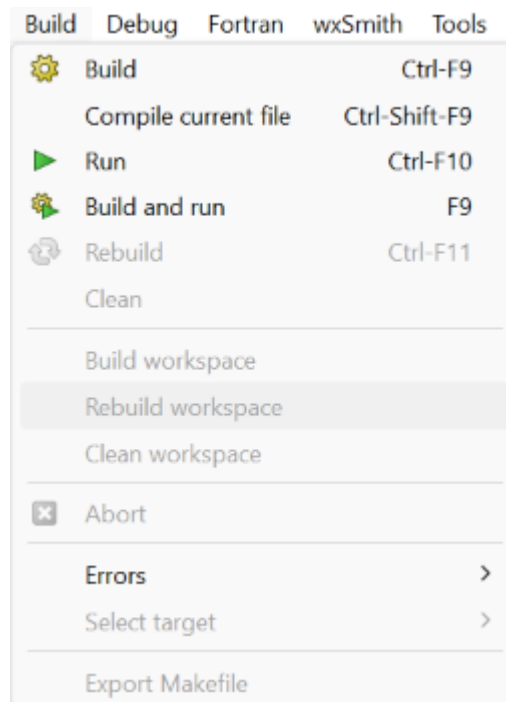
3. Hasil *Run* program



```
Haloo Worldd!!
Namaku Lanjar, Aku dari Blora dan hobiku adalah Ndoding(kali)
Aku berumur 15 tahun
Hasil perkalian: 200
Hasil pembagian: 5
```

5.3.3 Function pada C++

1. Lakukan *run and build* (logo segitiga hijau gear kuning) pada program untuk melihat hasilnya.



```
Hasil penjumlahan: 30
Hasil pengurangan: -10
Hasil perkalian: 200
Hasil pembagian: 5
Process returned 0 (0x0)   execution time : 3.371 s
Press any key to continue.
```

5.3.4 Method pada C++

1. Masukkan *source code* berikut pada program:

```
#include <iostream>
using namespace std;

class matematikal {
public:
    void penjumlahan(int a, int b) {
        int c = a + b;
        cout << "Hasil penjumlahan: " << c << endl;
    }

    void pengurangan() {
        int c = 10 - 20;
        cout << "Hasil pengurangan: " << c << endl;
    }
}
```

```

    }
};

class matematika2 {
public:
    int pembagian(int a, int b) {
        int c = a / b;
        return c;
    }

    int perkalian() {
        int c = 10 * 20;
        return c;
    }
};

int main() {

    matematika1 objek1;

    objek1.penjumlahan(10,20);

    objek1.pengurangan();

    matematika2 objek2;

    cout << "Hasil perkalian: " << objek2.perkalian()
<<endl;

    int d = objek2.pembagian(100,20);
    cout << "Hasil pembagian: " << d;

    return 0;
}turn 0;
}

```

2. Setelah *source code* dimasukkan, lakukan *run and build* (logo segitiga hijau gear kuning) pada program untuk melihat hasilnya.

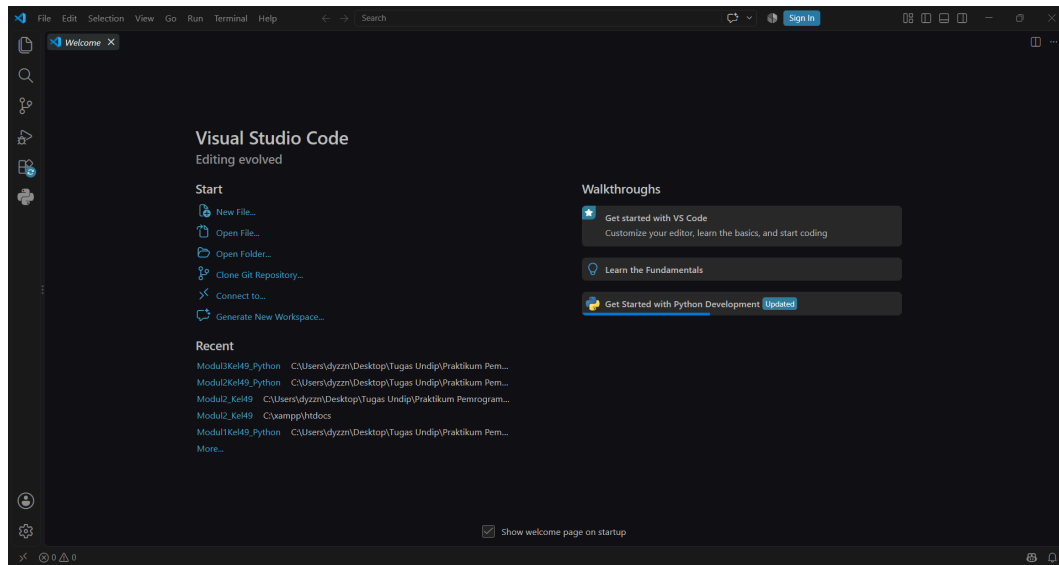
```

Hasil penjumlahan: 30
Hasil pengurangan: -10
Hasil perkalian: 200
Hasil pembagian: 5
Process returned 0 (0x0)   execution time : 4.444 s
Press any key to continue.
|

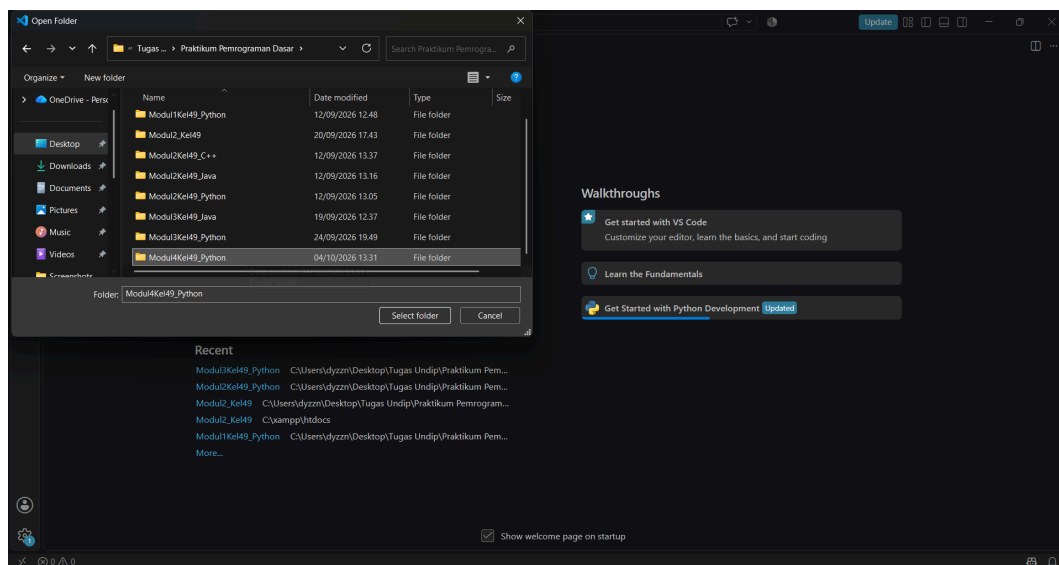
```

5.3.5 Function pada Python

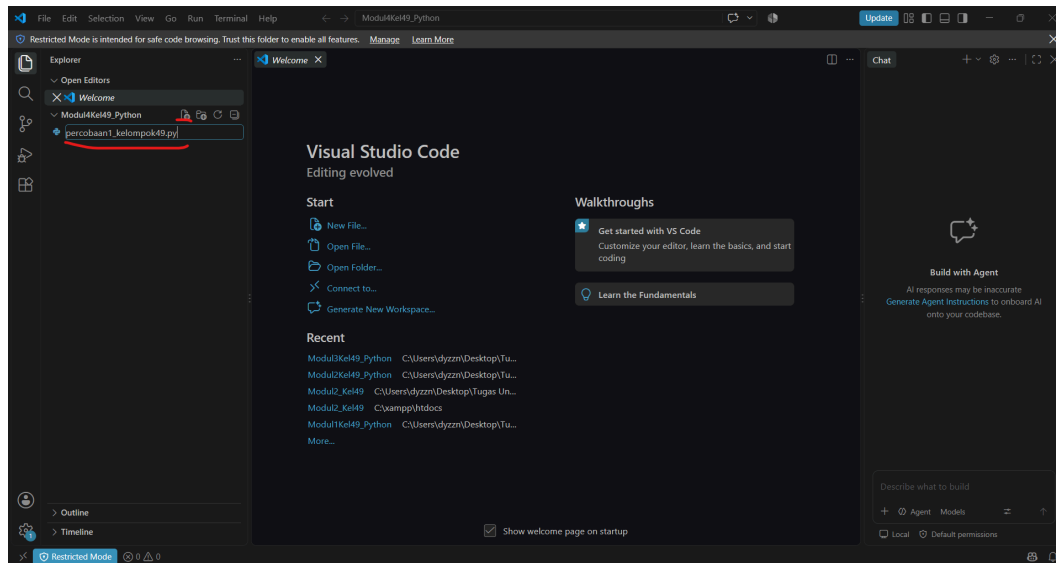
1. Pertama, buka *Visual Studio Code* untuk mengakses *Python* hingga menampilkan tampilan seperti di bawah ini.



2. Selanjutnya, buat *folder* baru dengan klik *Open Folder* dan beri nama folder Modul4KelXX_Python untuk tempat *project* baru



3. Selanjutnya, klik ikon yang di garis bawahhi itu untuk membuat percobaan1_kelompokXX.py



4. Jika sudah, silahkan masukkan *source code* berikut:

```
# 1. Non-return function (Void)
def sapa_pengunjung(nama, asal_daerah):
    print(
        f"Selamat datang {nama} dari {asal_daerah} di sistem "
        "pemantauan Desa Patakbanteng."
    )

# 2. Return function (Mengembalikan nilai yang sebenarnya)
def cek_status_suhu(suhu):
    print(f"Suhu yang tercatat: {suhu}°C")
    if suhu < 20:
        return "Status: Suhu terpantau dingin."
    else:
        return "Status: Suhu normal atau hangat."

# 3. Arbitrary function (Jumlah argumen bebas)
def catat_titik_cctv(*titik_lokasi):
    print("Lokasi CCTV yang sedang aktif:")
    for lokasi in titik_lokasi:
        print(f"- {lokasi}")

# 4. Anonymous function (Lambda)
# Misalnya untuk menghitung selisih data
hitung_selisih = lambda x, y: x - y

# Main program
sapa_pengunjung("Dafa", "Semarang")
print("-" * 30)

hasil_suhu = cek_status_suhu(18)
```

```

print(hasil_suhu)
print("-" * 30)

catat_titik_cctv("Gerbang Utama", "Balai Desa", "Area Parkir")
print("-" * 30)

print("Selisih pengunjung hari ini dan kemarin:",
hitung_selisih(150, 120))

```

5. Selanjutnya, jika berhasil maka akan muncul seperti tampilan di bawah ini

```

PS C:\Users\dyzzn\Desktop\Tugas Undip\Praktikum Pemrograman Dasar\Modul4Kel49_Python> & C:\Users\dyzzn\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/dyzzn/Desktop/Tugas Undip/Praktikum Pemrograman Dasar/Modul4Kel49_Python/percobaan1_kelompok49.py"
Selamat datang Dafa dari Semarang di sistem pemantauan Desa Patakbanteng.

-----
Suhu yang tercatat: 18°C
Status: Suhu terpantau dingin.
-----
Lokasi CCTV yang sedang aktif:
- Gerbang Utama
- Balai Desa
- Area Parkir
-----
Selisih pengunjung hari ini dan kemarin: 30
PS C:\Users\dyzzn\Desktop\Tugas Undip\Praktikum Pemrograman Dasar\Modul4Kel49_Python>

```

5.3.6 Method pada Python

1. Pada percobaan kali ini, silahkan buat *file* baru dengan *format* percobaan2_kelompokXX.py

```

24 # 4. Anonymous function (Lambda)
25 # Misalnya untuk menghitung selisih data
26 hitung_selisih = lambda x, y: x - y
27
28
29 # Main program
30
31 if __name__ == "__main__":
32     sapa_pengunjung("Adriani", "Tangerang")
33     print("-" * 30)
34
35     hasil_suhu = cek_status_suhu(18)

```

```

PS C:\Users\dyzzn\Desktop\Tugas Undip\Praktikum Pemrograman Dasar\Modul4Kel49_Python> & C:\Users\dyzzn\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/dyzzn/Desktop/Tugas Undip/Praktikum Pemrograman Dasar/Modul4Kel49_Python/percobaan1_kelompok49.py"
Selamat datang Adriani dari Tangerang di sistem pemantauan Desa Patakbanteng.

-----
Suhu yang tercatat: 18°C
Status: Suhu terpantau dingin.
-----
Lokasi CCTV yang sedang aktif:
- Gerbang Utama
- Balai Desa
- Area Parkir
-----
Selisih pengunjung hari ini dan kemarin: 30
PS C:\Users\dyzzn\Desktop\Tugas Undip\Praktikum Pemrograman Dasar\Modul4Kel49_Python>

```

2. Selanjutnya, silahkan masukkan *source code* berikut:

```
class MahasiswaTekkom:
```

```

def __init__(self, nama, konsentrasi):
    self.nama = nama
    self.konsentrasi = konsentrasi

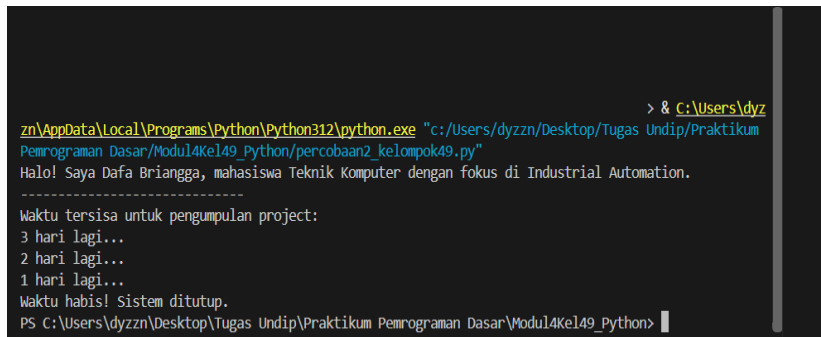
def profil(self):
    print(
        f"Halo! Saya {self.nama}, mahasiswa Teknik "
        f"Komputer dengan fokus di {self.konsentrasi}."
    )

def deadline_tugas(self, hari):
    print("Waktu tersisa untuk pengumpulan project:")
    while hari > 0:
        print(f"{hari} hari lagi...")
        hari -= 1
    print("Waktu habis! Sistem ditutup.")

# Main program
obj = MahasiswaTekkom("Dafa Briangga", "Industrial
Automation")
obj.profil()
print("-" * 30)
obj.deadline_tugas(3)

```

3. Jika berhasil, maka akan menampilkan tampilan seperti di bawah ini



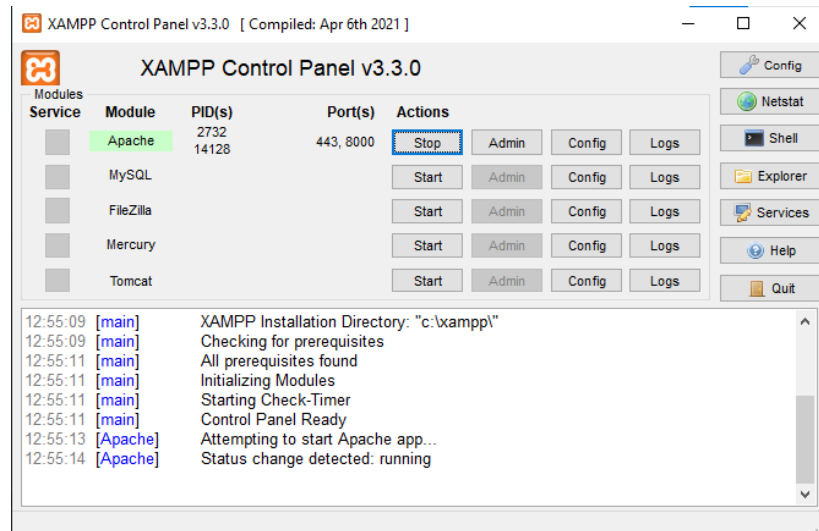
```

> & C:\Users\dyz
zn\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/dyzzn/Desktop/Tugas Undip/Praktikum
Pemrograman Dasar/Modul4Kel49_Python/percobaan2_kelompok49.py"
Halo! Saya Dafa Briangga, mahasiswa Teknik Komputer dengan fokus di Industrial Automation.
-----
Waktu tersisa untuk pengumpulan project:
3 hari lagi...
2 hari lagi...
1 hari lagi...
Waktu habis! Sistem ditutup.
PS C:\Users\dyzzn\Desktop\Tugas Undip\Praktikum Pemrograman Dasar\Modul4Kel49_Python>

```

5.3.7 Function pada PHP

1. Buka XAMPP *control panel* dan nyalakan *Apache*



2. Buat *folder* modul4PHP_kel49 dan buat *file* baru dengan nama `function.php` di dalam *folder* 'xampp/htdocs/modul4PHP_kel49'. Masukkan *source code* berikut pada *code editor*

```
<?php
function jumlahkan($a, $b) {
    return $a + $b;
}

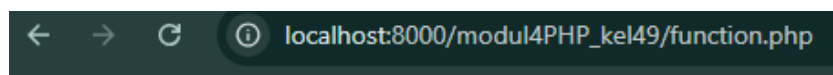
$x = 50;
$y = 30;

$hasil_return = jumlahkan($x, $y);

echo "Hasil function return: $hasil_return <br><hr>";
function cetak_jumlah($a, $b) {
    $hasil = $a + $b;
    echo "Hasil function non-return: $hasil";
}

cetak_jumlah($x, $y);
?>
```

3. Simpan *file* dan buka melalui *browser* dengan mengakses ke 'http://localhost:8000/modul4PHP_kel49/function.php'. dan akan muncul *Output* dari kode yang sudah dibuat



Hasil function return: 80

Hasil function non-return: 80

5.3.8 Method pada PHP

1. Buat *file* baru bernama `method.php` di dalam *folder* yang sama dengan percobaan *function* tadi. Lalu tambahkan *source code* berikut:

```
<?php
class MyClass {
    // Method dengan return type
    public function hitungAkar($nilai) {
        return sqrt($nilai);
    }

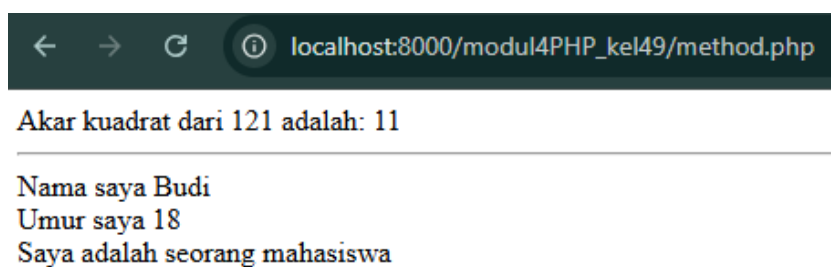
    // Method tanpa return type (menggabungkan data)
    public function cetakProfil($nama, $umur) {
        echo "Nama saya $nama <br>";
        echo "Umur saya $umur <br>";
        echo "Saya adalah seorang mahasiswa";
    }
}

// Pembuatan instance objek
$objek = new MyClass();

// Pemanggilan method menggunakan ->
echo "Akar kuadrat dari 121 adalah: " .
$objek->hitungAkar(121) . "<br><hr>";

$objek->cetakProfil("Budi", 18);
?>
```

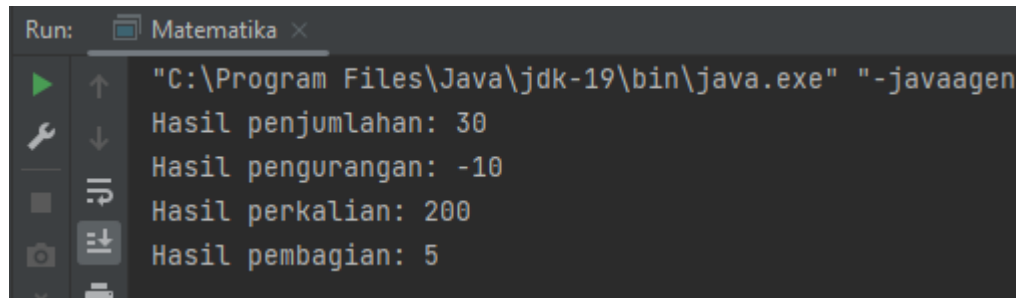
2. Simpan *file* dan buka melalui *browser* dengan mengakses ke `'http://localhost:8000/modul4PHP_kel49/method.php'`. dan akan muncul *Output* dari kode yang sudah dibuat



5.4 Hasil Percobaan dan Analisis

5.4.1 Function pada Java

1. Hasil *Run* program



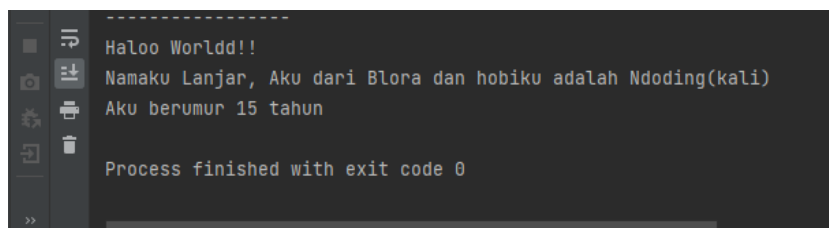
```
Run: Matematika x
"C:\Program Files\Java\jdk-19\bin\java.exe" "-javaagen
Hasil penjumlahan: 30
Hasil pengurangan: -10
Hasil perkalian: 200
Hasil pembagian: 5
```

2. Analisis

Class matematika memiliki empat *method*, yaitu *penjumlahan()*, *pengurangan()*, *pembagian()*, dan *perkalian()*. *Method* *penjumlahan()* menerima dua angka sebagai *parameter* lalu menjumlahkannya dan langsung menampilkan hasil. *Method* *pengurangan()* melakukan pengurangan angka 18 dan 24 tanpa menggunakan *parameter*. *Method* *pembagian()* menerima dua angka, melakukan pembagian, kemudian mengembalikan hasilnya dengan *return*. Sedangkan *method* *perkalian()* melakukan perkalian 18 dan 24 lalu mengembalikan hasilnya. Semua *method* tersebut kemudian dipanggil di dalam *main()* untuk menjalankan operasi matematika.

5.4.2 Method pada Java

1. Hasil *Run* program



```
-----
Halo Worldd!!
Namaku Lanjar, Aku dari Blora dan hobiku adalah Ndoding(kali)
Aku berumur 15 tahun

Process finished with exit code 0
```

2. Analisis

Method pada kode tersebut digunakan untuk menjalankan fungsi tertentu dalam *class intro*. *Method* sapa() digunakan untuk menampilkan sapaan, *method* perkenalan() digunakan untuk menggabungkan nama, kota, dan hobi menjadi sebuah kalimat, sedangkan *method* umur() digunakan untuk menampilkan umur. Setiap *method* dapat dipanggil dari *class* lain menggunakan *object* dari *class intro*. aku juga menjalankan di *main class* karena *intro* tidak dapat di *Run*.

5.4.3 Function pada C++

1. Hasil

```
Hasil penjumlahan: 30
Hasil pengurangan: -10
Hasil perkalian: 200
Hasil pembagian: 5
Process returned 0 (0x0)   execution time : 3.371 s
Press any key to continue.
|
```

2. Analisis

Program C++ ini menggunakan beberapa function untuk melakukan operasi matematika, yaitu penjumlahan, pengurangan, perkalian, dan pembagian. Function penjumlahan() dan pengurangan() langsung menampilkan hasil menggunakan cout, sedangkan function perkalian() dan pembagian() menggunakan return untuk mengembalikan hasil perhitungan. Pada fungsi main(), setiap function dipanggil untuk menjalankan proses dan menampilkan hasilnya. Program ini menunjukkan penggunaan function dengan parameter, tanpa parameter, serta function yang menggunakan return.

5.4.4 Method pada C++

1. Hasil

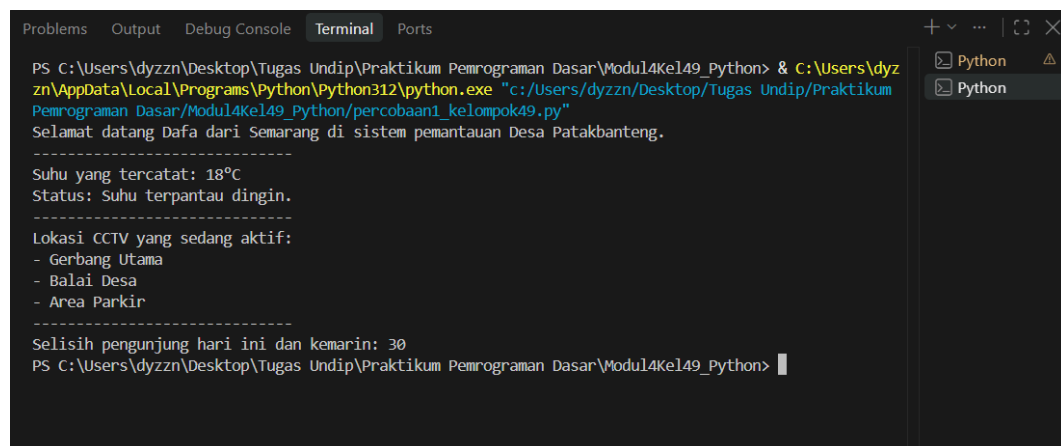
```
Hasil penjumlahan: 30
Hasil pengurangan: -10
Hasil perkalian: 200
Hasil pembagian: 5
Process returned 0 (0x0)   execution time : 4.444 s
Press any key to continue.
|
```

2. Analisis

Program C++ ini menggunakan dua class, yaitu matematika1 dan matematika2, yang berisi beberapa method untuk melakukan operasi matematika. Class matematika1 memiliki method penjumlahan() dengan parameter a dan b serta method pengurangan() tanpa parameter. Class matematika2 memiliki method perkalian() tanpa parameter dan pembagian() dengan parameter a dan b. Method yang menggunakan return akan mengembalikan hasil perhitungan untuk ditampilkan melalui cout. Pada fungsi main(), objek dari masing-masing class dibuat dan digunakan untuk memanggil setiap method. Program kemudian menampilkan hasil penjumlahan, pengurangan, perkalian, dan pembagian.

5.4.5 Function pada Python

1. Hasil



```
PS C:\Users\dyzn\Desktop\Tugas Undip\Praktikum Pemrograman Dasar\Modul4Kel49_Python> & C:\Users\dyzn\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/dyzn/Desktop/Tugas Undip/Praktikum Pemrograman Dasar/Modul4Kel49_Python/percobaan1_kelompok49.py"
Selamat datang Dafa dari Semarang di sistem pemantauan Desa Patakbanteng.
-----
Suhu yang tercatat: 18°C
Status: Suhu terpantau dingin.
-----
Lokasi CCTV yang sedang aktif:
- Gerbang Utama
- Balai Desa
- Area Parkir
-----
Selisih pengunjung hari ini dan kemarin: 30
PS C:\Users\dyzn\Desktop\Tugas Undip\Praktikum Pemrograman Dasar\Modul4Kel49_Python>
```

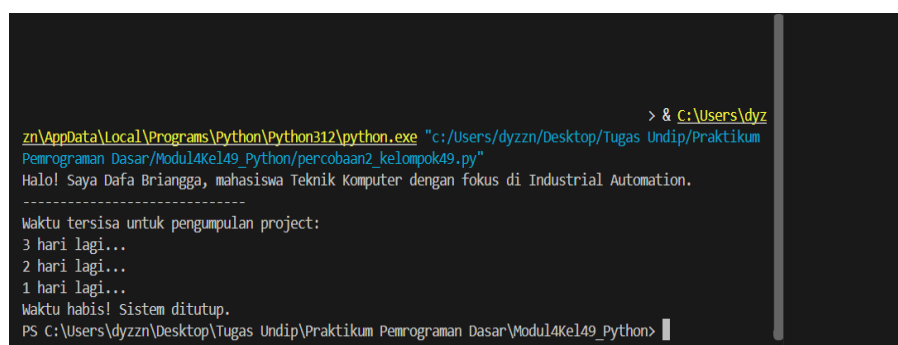
2. Analisis

Pada *source code* tersebut, program diawali dengan mendefinisikan fungsi *non-return* atau *void* bernama *sapa_pengunjung* yang menerima parameter nama dan *asal_daerah* untuk mencetak pesan sambutan tanpa mengembalikan nilai ke memori. Selanjutnya, program beralih ke fungsi *return* *cek_status_suhu* yang memanfaatkan struktur pengkondisian *if-else* untuk mengevaluasi parameter suhu. Jika angkanya di bawah 20, program akan mengembalikan tipe data *string* dengan status dingin, dan jika tidak, mengembalikan status normal. Kemudian, program memproses fungsi *arbitrary* *catat_titik_cctv* berbekal parameter dinamis *titik_lokasi*, di mana perulangan *for* digunakan untuk mengurai dan mencetak berapa pun jumlah lokasi yang diinputkan secara berurutan.

Selain itu, program juga menyertakan fungsi anonim atau *lambda* pada variabel *hitung_selisih* guna menjalankan operasi pengurangan secara ringkas. Pada bagian akhir, seluruh rangkaian fungsi tersebut dieksekusi secara berurutan di dalam blok utama program. Hasil dari fungsi *return* dan kalkulasi *lambda* langsung ditangkap dan ditampilkan ke layar, diselingi perintah *print* pemisah sebagai garis batas untuk merapikan visual *output* terminal.

5.4.6 Method pada Python

1. Hasil



```
> & C:\Users\dyz
zn\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/dyzzn/Desktop/Tugas Undip/Praktikum
Pemrograman Dasar/Modul4Kel49_Python/percobaan2_kelompok49.py"
Halo! Saya Dafa Briangga, mahasiswa Teknik Komputer dengan fokus di Industrial Automation.
-----
waktu tersisa untuk pengumpulan project:
3 hari lagi...
2 hari lagi...
1 hari lagi...
waktu habis! Sistem ditutup.
PS C:\Users\dyzzn\Desktop\Tugas Undip\Praktikum Pemrograman Dasar\Modul4Kel49_Python>
```

2. Analisis

Pada *source code* tersebut, program diawali dengan mendefinisikan sebuah *class* bernama *MahasiswaTekkom* yang merepresentasikan cetak

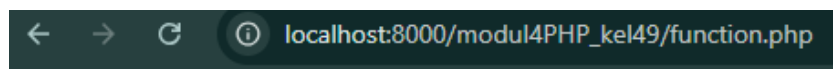
biru dari sebuah objek. Di dalam *class* ini, terdapat metode *constructor* bernama *init* yang berfungsi untuk menginisialisasi atribut objek, yaitu nama dan konsentrasi, pada saat objek baru pertama kali dibuat. Selanjutnya, program memiliki metode *non-return* atau *void* bernama *profil* yang bertugas mencetak teks pengenalan ke layar dengan memanggil data dari atribut yang telah tersimpan sebelumnya.

Setelah itu, terdapat juga metode *deadline_tugas* yang menggunakan struktur perulangan *while* untuk memproses hitung mundur sisa hari pengumpulan proyek. Selama nilai parameter hari lebih besar dari nol, kondisi perulangan akan bernilai *true* sehingga program terus mencetak sisa waktu dan mengurangi nilainya secara bertahap menggunakan operasi *decrement* hari $-= 1$ hingga kondisi pada akhirnya berubah menjadi *false*.

Pada bagian akhir, program mengeksekusi blok utama dengan membentuk objek bernama *obj* yang diberikan argumen teks Dafa Briangga dan *Industrial Automation*. Program kemudian memanggil kedua metode dari objek tersebut secara berurutan, yang diselingi dengan pemanggilan fungsi *print* pemisah berupa garis batas untuk merapikan baris *output* akhir.

5.4.7 Function pada PHP

1. Hasil program



Hasil function return: 80

Hasil function non-return: 80

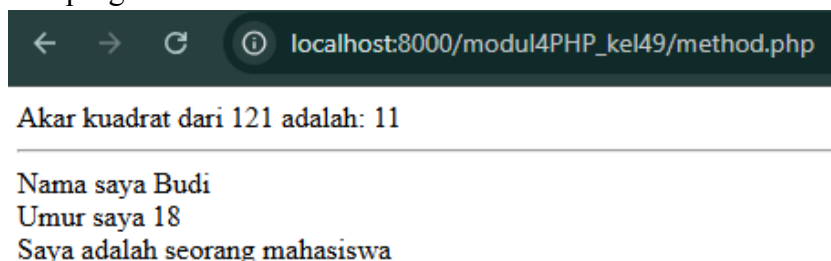
2. Analisis

Program ini diawali dengan mendefinisikan sebuah fungsi bernama *jumlahkan()* dengan parameter lokal *\$a* dan *\$b*. Fungsi ini bertugas menjumlahkan nilai dari kedua parameter tersebut dan mengembalikan hasilnya ke luar fungsi menggunakan perintah *return*. Selanjutnya, program menginisialisasi variabel luar bernama *\$x* yang diisi dengan angka 50 dan variabel

`$y` yang diisi dengan angka 30. Nilai dari variabel `$x` dan `$y` ini kemudian diteruskan sebagai argumen untuk ditampung ke dalam parameter `$a` dan `$b` pada saat memanggil fungsi `jumlahkan()`. Hasil kembalian dari fungsi tersebut (yaitu angka 80) kemudian disimpan ke dalam sebuah variabel penampung bernama `$hasil_return` untuk ditampilkan ke layar menjadi kalimat *output* "Hasil function return: 80" beserta tag HTML `<br><hr>` menggunakan perintah `echo`. Setelah itu, program juga mendefinisikan fungsi kedua bernama `cetak_jumlah()` yang menggunakan parameter `$a` dan `$b` serupa, namun fungsi ini menggunakan *non-return*, yang langsung memproses hasil penjumlahannya ke dalam variabel lokal bernama `$hasil` dan mencetaknya secara langsung dari dalam blok fungsi. Pada akhir baris eksekusi, program memanggil fungsi `cetak_jumlah()` dengan kembali meneruskan nilai dari variabel `$x` dan `$y` untuk mengeksekusi dan menghasilkan teks *output* "Hasil function non-return: 80" secara langsung ke layar.

5.4.8 Method pada PHP

1. Hasil program



2. Analisis

Program ini dengan mendefinisikan sebuah *class* bernama `MyClass`. Di dalam kelas tersebut, program mendefinisikan *method* pertama bernama `hitungAkar()` dengan parameter lokal `$nilai`. *Method* ini bertugas menghitung nilai akar kuadrat dari parameter tersebut menggunakan fungsi bawaan `sqrt()` dan mengembalikan hasilnya ke luar fungsi menggunakan perintah `return`. Selanjutnya, program juga mendefinisikan *method* kedua bernama `cetakProfil()` dengan dua parameter lokal, yakni `$nama` dan `$umur`. Berbeda dengan *method* sebelumnya, *method* ini menggunakan *non-return*, yaitu langsung

memproses data dari kedua parameter tersebut untuk dicetak ke dalam beberapa baris teks menggunakan perintah `echo`.

Pada blok eksekusi utama, program membuat sebuah instansiasi objek (*instance*) dari `MyClass` yang kemudian disimpan ke dalam variabel bernama `$objek`. Untuk menggunakan fungsi di dalam kelas tersebut, program memanggil *method* `hitungAkar()` dari variabel `$objek` menggunakan operator `->` dengan meneruskan angka 121 sebagai argumen yang ditampung ke dalam parameter `$nilai`. Hasil kembalian dari perhitungan akar kuadrat tersebut (yaitu angka 11) kemudian disambungkan dengan teks awal beserta *tag HTML* menggunakan operator titik (`.`), sehingga menghasilkan *output* kalimat "Akar kuadrat dari 121 adalah: 11" yang dicetak ke layar beserta garis bawah `<hr>` menggunakan perintah `echo`.

Pada akhir baris eksekusi, program memanggil *method* `cetakProfil()` dari objek yang sama dengan meneruskan teks "Budi" dan angka 18 sebagai argumen untuk ditampung ke parameter `$nama` dan `$umur`. Pemanggilan ini akan langsung mengeksekusi perintah `echo` di dalam blok fungsi guna menghasilkan teks *output* secara berurutan, yaitu "Nama saya Budi", "Umur saya 18", dan "Saya adalah seorang mahasiswa" langsung ke layar tanpa memerlukan kembalian data.

5.5 Tugas Praktikum (Source code diketik, diberi SS hasil, dan di analisa)

A. Kevin Teopilus Tarigan

1. Source code
2. Running program
3. Analisis

B. Adilla Achmad Zaky (Pyhton)

1. Source code

```
def judul():
    return "Data Kelas Mahasiswa by Adilla Zaky KELOMPOK 49 SHIFT 8"

def hitung_total_mahasiswa(jml_a, jml_b, jml_c, jml_d):
    return jml_a + jml_b + jml_c + jml_d

class KelasKuliah:
    def __init__(self, nama_kelas, jumlah_mahasiswa):
        self.nama_kelas = nama_kelas
        self.jumlah_mahasiswa = jumlah_mahasiswa

    def tampilkan_info(self):
        print("\n" + "=" * 30)
        print(f"KELAS          : Kelas {self.nama_kelas}")
        print(f"JUMLAH MAHASISWA: {self.jumlah_mahasiswa} orang")
        print("=" * 30)

if __name__ == "__main__":
    print(judul() + "\n")

    kelas_a = KelasKuliah("A", 35)
    kelas_b = KelasKuliah("B", 42)
    kelas_c = KelasKuliah("C", 28)
    kelas_d = KelasKuliah("D", 20)

    sistem_jalan = True

    while sistem_jalan:
        print("\nDATA KELAS KULIAH")
        print("1. Kelas A")
        print("2. Kelas B")
        print("3. Kelas C")
        print("4. Kelas D")
        print("5. Total Seluruh Mahasiswa")
        print("6. Keluar")

        pilihan = input("Pilih menu (1-6): ")

        if pilihan == "1":
            kelas_a.tampilkan_info()
        elif pilihan == "2":
            kelas_b.tampilkan_info()
```

```

elif pilihan == "3":
    kelas_c.tampilkan_info()
elif pilihan == "4":
    kelas_d.tampilkan_info()
elif pilihan == "5":
    total = hitung_total_mahasiswa(
        kelas_a.jumlah_mahasiswa,
        kelas_b.jumlah_mahasiswa,
        kelas_c.jumlah_mahasiswa,
        kelas_d.jumlah_mahasiswa,
    )
    print(f"Total Seluruh Mahasiswa (Kelas A - D): {total}
orang")
elif pilihan == "6":
    print("\nDone.....")
    sistem_jalan = False
else:
    print("\nGada pilihannya! Yang bener! Pilih angka 1 sampai
6.")

```

2. Running program

```

PS C:\coba_clone\Adilla-Achmad-Zaky_KELompok49> & "C:\Users\HP Pa
ELompok49\tugas.py
Data Kelas Mahasiswa by Adilla Zaky KELOMPOK 49 SHIFT 8

DATA KELAS KULIAH
1. Kelas A
2. Kelas B
3. Kelas C
4. Kelas D
5. Total Seluruh Mahasiswa
6. Keluar
Pilih menu (1-6): 1

=====
KELAS          : Kelas A
JUMLAH MAHASISWA: 35 orang
=====

DATA KELAS KULIAH
1. Kelas A
2. Kelas B
3. Kelas C
4. Kelas D
5. Total Seluruh Mahasiswa
6. Keluar
Pilih menu (1-6): 6

Done.....
PS C:\coba_clone\Adilla-Achmad-Zaky_KELompok49>

```

3. Analisis

Pada *source code* tersebut, program diawali dengan pendefinisian fungsi `judul()` untuk mengembalikan teks judul program, serta fungsi

`hitung_total_mahasiswa()` untuk menjumlahkan parameter angka yang dimasukkan. Program juga mendefinisikan sebuah *class* dengan nama `KelasKuliah` yang memiliki metode inisialisasi `__init__()` untuk menyimpan atribut `nama_kelas` dan `jumlah_mahasiswa`, beserta metode `tampilkan_info()` untuk mencetak rincian data kelas tersebut ke layar. Pada blok eksekusi utama, program memanggil fungsi `judul()` dan menginisialisasi empat buah objek dari kelas tersebut, mulai dari objek `kelas_a` hingga `kelas_d`. Selanjutnya, program membuat sebuah variabel bernama `sistem_jalan` yang diisi dengan nilai *true* sebagai syarat kondisi berjalannya perulangan *while*. Selama perulangan tersebut bernilai *true*, program akan terus menampilkan daftar menu dan meminta pengguna memasukkan angka yang nantinya akan disimpan dalam format teks ke variabel `pilihan`. Nilai pada variabel `pilihan` tersebut kemudian dievaluasi menggunakan struktur pengkondisian *if-elif-else*; jika nilainya adalah "1", "2", "3", atau "4", program akan memanggil metode `tampilkan_info()` dari objek kelas yang bersesuaian. Jika memilih "5", fungsi `hitung_total_mahasiswa()` akan dieksekusi untuk menampilkan total mahasiswa. Namun, jika pengguna menginputkan angka "6", program akan mengubah nilai variabel `sistem_jalan` menjadi *false* sehingga perulangan utama *while* akan berhenti secara otomatis. Terakhir, apabila *input* pengguna pada variabel `pilihan` tidak memenuhi seluruh kondisi tersebut (semuanya bernilai *false*), maka program akan mengeksekusi blok alternatif *else* untuk menampilkan pesan peringatan ke layar.

link repo:

https://github.com/Adillazaky08/Adilla-Achmad-Zaky_KELOMPOK49

C. **Adriann Dielandoro**

1. *Source code*

```
#include <iostream>
#include <string>

using namespace std;

string getWatermark() {
    return
```

```

"=====\n"
"        SISTEM PEMESANAN WARUNG MAKAN - KELOMPOK
49\n"

"=====";
}

double hitungTotalBayar(double hargaPerPorsi, int
jumlahPesan) {
    double total = hargaPerPorsi * jumlahPesan;

    if (jumlahPesan >= 3) {
        return total * 0.95;
    }
    return total;
}

class WarungMakan {
private:
    string daftarMenu[3] = {
        "Nasi Goreng Spesial", "Mie Goreng Jawa", "Ayam
Bakar Madu"
    };
    double daftarHarga[3] = {25000, 20000, 30000};
    int stokPorsi[3] = {20, 15, 10};

public:
    void tampilkanMenu() {
        cout << "\n--- DAFTAR MENU WARUNG ---" << endl;
        for (int i = 0; i < 3; i++) {
            cout << i + 1 << ". " << daftarMenu[i] << " -
Rp" << daftarHarga[i]
                << " (Sisa: " << stokPorsi[i] << " porsi)"
<< endl;
        }
    }

    void pesanMakanan(int indeksMenu, int jumlahPesan) {
        int indeks = indeksMenu - 1;

        if (indeks < 0 || indeks >= 3) {
            cout << "Gagal: Pilihan menu tidak valid!" <<
endl;
        } else if (jumlahPesan > stokPorsi[indeks]) {
            cout << "Gagal: Sisa porsi " <<
daftarMenu[indeks]
                << " tidak mencukupi. Sisa: " <<
stokPorsi[indeks] << endl;
        } else {
            stokPorsi[indeks] -= jumlahPesan;

            double tagihan =
hitungTotalBayar(daftarHarga[indeks], jumlahPesan);

            cout << "Sukses: " << jumlahPesan << " porsi "

```

```

<< daftarMenu[indeks]
    << " sedang disiapkan." << endl;
    cout << "Total Tagihan (termasuk diskon jika
ada): Rp" << tagihan
    << endl;
}
}
};

int main() {
    cout << getWatermark() << endl << endl;

    WarungMakan warung;
    int pilihan = 0;

    while (pilihan != 3) {
        cout << "\n=== MENU UTAMA ===" << endl;
        cout << "1. Tampilkan Daftar Menu" << endl;
        cout << "2. Pesan Makanan" << endl;
        cout << "3. Keluar" << endl;
        cout << "Pilihan Anda (1-3): ";
        cin >> pilihan;

        if (pilihan == 1) {
            warung.tampilkanMenu();
        } else if (pilihan == 2) {
            int menuPilihan, porsi;
            cout << "\nMasukkan nomor menu (1-3): ";
            cin >> menuPilihan;
            cout << "Masukkan jumlah porsi: ";
            cin >> porsi;

            if (porsi > 0) {
                warung.pesanMakanan(menuPilihan, porsi);
            } else {
                cout << "Input tidak valid! Pesanan minimal
1 porsi." << endl;
            }
        } else if (pilihan == 3) {
            cout << "\nTerima kasih, selamat menikmati
hidangan Anda!" << endl;
        } else {
            cout << "\nPilihan tidak dikenali, silakan coba
lagi!" << endl;
        }
    }

    return 0;
}

```

2. Running program

```
"C:\Users\dyzzn\Desktop\Tug...
=====
SISTEM PEMESANAN WARUNG MAKAN - KELOMPOK XX
=====

=== MENU UTAMA ===
1. Tampilkan Daftar Menu
2. Pesan Makanan
3. Keluar
Pilihan Anda (1-3): 1

--- DAFTAR MENU WARUNG ---
1. Nasi Goreng Spesial - Rp25000 (Sisa: 20 porsi)
2. Mie Goreng Jawa - Rp20000 (Sisa: 15 porsi)
3. Ayam Bakar Madu - Rp30000 (Sisa: 10 porsi)

=== MENU UTAMA ===
1. Tampilkan Daftar Menu
2. Pesan Makanan
3. Keluar
Pilihan Anda (1-3): 2

Masukkan nomor menu (1-3): 2
Masukkan jumlah porsi: 13
Sukses: 13 porsi Mie Goreng Jawa sedang disiapkan.
Total Tagihan (termasuk diskon jika ada): Rp247000

=== MENU UTAMA ===
1. Tampilkan Daftar Menu
2. Pesan Makanan
3. Keluar
Pilihan Anda (1-3): 3

Terima kasih, selamat menikmati hidangan Anda!

Process returned 0 (0x0)   execution time : 13.302 s
Press any key to continue.
```

3. Analisis

Pada *source code* tersebut, program diawali dengan pendefinisian fungsi `getWatermark()` untuk mengembalikan teks *watermark*/judul program kelompok, serta fungsi `hitungTotalBayar()` berparameter untuk menghitung total tagihan sekaligus mengevaluasi syarat diskon. Program juga mendefinisikan sebuah *class* dengan nama `WarungMakan` yang menggunakan struktur *array* untuk menyimpan daftar menu, harga, dan ketersediaan porsi. *Class* ini memiliki metode `tampilkanMenu()` untuk mencetak rincian daftar makanan menggunakan perulangan *for*, beserta metode `pesanMakanan()` berparameter untuk mengevaluasi ketersediaan porsi, mengurangi stok *array*, dan memanggil fungsi `hitungTotalBayar()`. Pada blok eksekusi utama (*main*), program memanggil fungsi `getWatermark()` dan menginisialisasi sebuah objek dari kelas tersebut dengan nama `warung`.

Selanjutnya, program membuat sebuah variabel *integer* bernama `pilihan` yang diisi dengan nilai awal 0 sebagai syarat kondisi berjalannya perulangan *while*. Selama variabel `pilihan` tidak sama dengan 3 (kondisi bernilai *true*), program akan terus menampilkan menu antarmuka utama dan meminta pengguna memasukkan angka yang nantinya akan disimpan ke dalam variabel `pilihan`. Nilai pada variabel `pilihan` tersebut kemudian dievaluasi menggunakan struktur pengkondisian *if-else if-else*; jika nilainya adalah 1, program akan memanggil metode `tampilkanMenu()` dari objek `warung`. Jika memilih 2, program akan

meminta input nomor menu dan jumlah porsi, lalu memanggil metode `pesanMakanan()`.

Namun, jika pengguna memasukkan angka 3, program akan menampilkan teks ucapan terima kasih dan nilai variabel `pilihan` akan membuat kondisi perulangan utama *while* menjadi *false* sehingga program berhenti secara otomatis. Terakhir, apabila input pengguna pada variabel `pilihan` tidak memenuhi seluruh kondisi tersebut (bernilai *false* untuk 1, 2, dan 3), maka program akan mengeksekusi blok alternatif *else* untuk menampilkan pesan peringatan input tidak valid ke layar.

link repo:

https://github.com/Adriann249/Adriann-Dielandoro_KELOMPOK49

D. Andreas Hasiholan Brilliantino Sianturi

1. Source code

```
#include <iostream>
#include <string>

using namespace std;

string getJudul() {
    return "=== SISTEM LOST & FOUND KAMPUS ===";
}

string cekPrioritas(string kategori) {
    if (kategori == "Elektronik") {
        return "Tinggi";
    } else if (kategori == "Dokumen") {
        return "Tinggi";
    } else {
        return "Normal";
    }
}

class LostFound {
private:
    string namaBarang[5] = {
        "Dompet Hitam",
        "Flashdisk Sandisk",
        "Kartu Mahasiswa",
        "Payung Biru",
        "Headset"
    };

    string kategori[5] = {
        "Aksesoris",
        "Elektronik",
        "Dokumen",
    };
};
```

```

        "Aksesoris",
        "Elektronik"
    };

    bool sudahDiambil[5] = {
        false, false, false, false, false
    };

    int jumlahBarang = 5;

public:
    void tampilkanBarang() {
        cout << "\n--- DAFTAR BARANG DITEMUKAN ---" <<
endl;

        for (int i = 0; i < jumlahBarang; i++) {
            cout << i + 1 << ". " << namaBarang[i]
                << " | " << kategori[i];

            if (sudahDiambil[i]) {
                cout << " | Sudah diambil";
            } else {
                cout << " | Masih tersedia";
            }

            cout << endl;
        }
    }

    void cariBarang(string kata) {
        bool ditemukan = false;

        cout << "\n--- HASIL PENCARIAN ---" << endl;

        for (int i = 0; i < jumlahBarang; i++) {
            if (namaBarang[i].find(kata) != string::npos) {
                cout << "Barang ditemukan: " <<
namaBarang[i] << endl;
                cout << "Kategori: " << kategori[i] <<
endl;

                if (sudahDiambil[i]) {
                    cout << "Status: Sudah diambil" <<
endl;
                } else {
                    cout << "Status: Masih tersedia" <<
endl;
                }

                cout << "Prioritas: "
                    << cekPrioritas(kategori[i]) << endl;
            }
        }
    }

```

```

        ditemukan = true;
    }
}

if (!ditemukan) {
    cout << "Barang tidak ditemukan." << endl;
}

// Method non-return dengan parameter
void ambilBarang(int nomor) {
    int indeks = nomor - 1;

    if (indeks < 0 || indeks >= jumlahBarang) {
        cout << "Nomor barang tidak valid." << endl;
    }
    else if (sudahDiambil[indeks]) {
        cout << "Barang tersebut sudah diambil
sebelumnya." << endl;
    }
    else {
        sudahDiambil[indeks] = true;

        cout << "Barang \"" << namaBarang[indeks]
        << "\" berhasil ditandai sebagai sudah
diambil."
        << endl;
    }
}

};

int main() {

    LostFound sistem;
    int pilihan = 0;

    cout << getJudul() << endl;

    while (pilihan != 4) {

        cout << "\n=== MENU UTAMA ===" << endl;
        cout << "1. Lihat Barang Ditemukan" << endl;
        cout << "2. Cari Barang" << endl;
        cout << "3. Ambil Barang" << endl;
        cout << "4. Keluar" << endl;
        cout << "Pilihan: ";
        cin >> pilihan;

        if (pilihan == 1) {

            sistem.tampilkanBarang();

        }
        else if (pilihan == 2) {

```

```

        string kata;

        cout << "Masukkan nama barang yang dicari: ";
        cin.ignore();
        getline(cin, kata);

        sistem.cariBarang(kata);

    }
    else if (pilihan == 3) {

        int nomor;

        sistem.tampilkanBarang();

        cout << "\nMasukkan nomor barang yang ingin
diambil: ";
        cin >> nomor;

        sistem.ambilBarang(nomor);

    }
    else if (pilihan == 4) {

        cout << "\nTerima kasih telah menggunakan
sistem."
        << endl;

    }
    else {

        cout << "\nPilihan tidak tersedia." << endl;

    }

}

return 0;
}

```

2. *Running program*

```

=== SISTEM LOST & FOUND KAMPUS ===

=== MENU UTAMA ===
1. Lihat Barang Ditemukan
2. Cari Barang
3. Ambil Barang
4. Keluar
Pilihan: 1

--- DAFTAR BARANG DITEMUKAN ---
1. Dompot Hitam | Aksesoris | Masih tersedia
2. Flashdisk Sandisk | Elektronik | Masih tersedia
3. Kartu Mahasiswa | Dokumen | Masih tersedia
4. Payung Biru | Aksesoris | Masih tersedia
5. Headset | Elektronik | Masih tersedia

=== MENU UTAMA ===
1. Lihat Barang Ditemukan
2. Cari Barang
3. Ambil Barang
4. Keluar
Pilihan: 2
Masukkan nama barang yang dicari: Headset

--- HASIL Pencarian ---
Barang ditemukan: Headset
Kategori: Elektronik
Status: Masih tersedia
Prioritas: Tinggi

```

3. Analisis

Program *Sistem Lost & Found Kampus* dibuat untuk membantu mencatat dan mencari barang yang ditemukan di lingkungan kampus. Program ini menggunakan *array* untuk menyimpan nama dan kategori barang, pengkondisian untuk menentukan status dan prioritas barang, serta perulangan untuk menampilkan dan mencari data barang. Program juga menggunakan *function* dan *method*. *Function* digunakan dengan *return type*, yaitu *getJudul()* tanpa parameter dan *cekPrioritas()* dengan parameter. Sedangkan *method* menggunakan *non-return type* atau *void*, seperti *tampilkanBarang()*, *cariBarang()*, dan *ambilBarang()*. Dengan demikian, program ini telah menerapkan materi pengkondisian, perulangan, *function*, *method*, *return type* berparameter dan tanpa parameter, serta *non-return type*.

link repo:

https://github.com/andresalto/Andreas-Hasiholan_KELOMPOK49

5.6 Kesimpulan

1. Kelompok kami memahami hubungan antara abstraksi dan enkapsulasi dalam pemrograman berorientasi objek melalui penggunaan method. Dengan method, bagian logika program yang kompleks dapat dipisahkan dari bagian yang digunakan oleh pengguna, sedangkan access modifier digunakan untuk membatasi akses terhadap data tertentu.
2. Kelompok kami dapat membedakan function dan method berdasarkan cara penggunaannya. Function dapat digunakan secara mandiri, sedangkan method merupakan bagian dari objek dan dapat menggunakan data atau keadaan yang dimiliki oleh objek tersebut.
3. Kelompok kami mulai menerapkan clean code dengan membagi program menjadi beberapa method berdasarkan tugasnya masing-masing. Hal ini membuat kode lebih terstruktur dan mengurangi pengulangan kode yang sama.
4. Kelompok kami memahami bahwa penggunaan method dapat membuat program lebih mudah dibaca dan diperbaiki karena setiap bagian memiliki tugas tertentu. Namun, penggunaan method yang terlalu banyak atau saling bergantung juga perlu diperhatikan agar struktur program tetap sederhana dan mudah dipahami.
5. Kelompok kami dapat menerapkan konsep method pada beberapa bahasa pemrograman dengan memperhatikan parameter, access modifier, dan return type yang digunakan. Meskipun sintaks setiap bahasa berbeda, konsep dasarnya tetap dapat diterapkan dengan cara yang serupa.

Ketentuan Penulisan Format Praktikum

1. Format Penulisan :
 - a) Margin : 4433 (kiri, atas, kanan, bawah)
 - b) Ukuran kertas : A4
 - c) Menggunakan *font Times New Roman* ukuran 12pt warna hitam
 - d) Jarak antar baris tiap paragraf 1.5 dengan spasi *after* dan *before* : 0pt
 - e) Paragraf menggunakan *justify*
 - f) Semua kata yang tidak ada KBBI atau istilah asing diketik miring (*italic*)
 - g) Kata dan atau kalimat yang berkaitan dengan kode program, diletakkan di dalam kotak (**tabel, bukan shape**), diketik dengan ketentuan:
 - Font Style: Courier New
 - Font Size: 10,
 - Line Spacing: 1 (before = 0, after = 0)
 - Font Color: Hitam
 - h) Jarak antar baris di *source code* 1.0
 - i) Keterangan gambar menggunakan fitur *Insert Caption* yang ada di tab *References* , menggunakan font times new roman ukuran 10 warna hitam.
 - j) Jarak antar gambar ke keterangan 1.0, rata tengah (center), tidak perlu di-**bold**
 - k) Penulisan sumber menggunakan font times new roman ukuran 10, rata kanan.
 - l) Tiap sub-bab ganti halaman (menggunakan *page break*) dan tiap sub-sub-bab diberi jarak satu *Enter*.
 - m) Judul sub-bab diterapkan heading runtut dan sejajarkan.
2. **Apabila ketahuan *Copy Paste* atau *Download Edit***, maka akan disuruh membuat laporan baru dan jika tidak membuat laporan baru, maka **nilai laporan= 0**.

Catatan:

1. Laporan asistensi dikirim via E-mail dengan nama file (.docx) dan subjek email: PROGDAS2026_Modul4_KelXX
2. Konfirmasi apabila sudah mengirim laporan melalui WA.
3. Dilarang copy paste atau nilai dibagi 2 (baik yang copy maupun yang di-copy)
4. Batas pengumpulan laporan: Senin, 5 Oktober 2026 23.59
5. Batas asistensi (pengumpulan laporan hingga mendapat AC): Kamis, 22 Oktober 2026 23.59
6. Melewati batas pengumpulan atau deadline ac nilai laporan dikurangi 15 perhari
7. Semua Kelompok Ganjil mengumpulkan Laporan ke email setiawanlanjar2@gmail.com (Lanjar)
8. Semua Kelompok Genap mengumpulkan Laporan ke email habibtrmojo@gmail.com (Habib)

Kontak Konfirmasi Asisten Modul 4

Lanjar Setiawan >> WA: 08979213974

Habib Mukhlis Triatmojo >> WA: 085702213530